

Project Title	InsureAI – An End-to-End AI Assistant for the Insurance Industry (ML • DL • CNN • RNN • Generative AI • Prompt Engineering • LLM APIs)
Skills take away From This Project	Python, Data Cleaning & EDA, Machine Learning (Classification & Regression), Deep Learning (ANN), CNN for Image Classification, RNN/LSTM for Text, Generative AI, Prompt Engineering, LLM API Integration, Streamlit App Development, Model Evaluation & Deployment
Domain	Insurance (Health, Vehicle & General Insurance)

Problem Statement:

Insurance companies handle thousands of customers, claims, documents, and images every day. Manual processing leads to slow claim settlement, undetected fraud, poor customer experience, and high operational cost.

In this project, learners will build “InsureAI” — a single Streamlit application for an insurance company that contains five AI modules, one for each core topic of the course:

- Module 1 (Machine Learning): Predict the medical insurance premium of a customer using regression, and detect whether an insurance claim is fraudulent using classification.
- Module 2 (Deep Learning – ANN): Build an Artificial Neural Network on the same tabular data and compare its performance with the ML models.
- Module 3 (CNN): Classify vehicle images as “damaged” or “not damaged” so that vehicle claims can be verified automatically from uploaded photos.
- Module 4 (RNN / LSTM): Analyse the sentiment of customer reviews/complaints about the insurance company (positive / negative) using an LSTM model.
- Module 5 (Generative AI + Prompt Engineering + LLM API): Build an insurance chatbot that answers policy-related questions, summarises claim descriptions, and drafts customer reply emails by calling an LLM API (e.g., OpenAI / Gemini / Groq) with well-designed prompts.

All five modules must be integrated into one simple Streamlit web application with a separate page/tab for each module.

Business Use Cases:

- Premium Prediction: Help sales teams give instant, data-driven premium quotes to new customers based on age, BMI, smoking habits, etc.
- Fraud Detection: Automatically flag suspicious claims for investigation, reducing financial losses from fraudulent payouts.
- Automated Damage Verification: Let customers upload vehicle photos during claim filing; the CNN verifies visible damage and speeds up claim approval.
- Customer Sentiment Monitoring: Track customer satisfaction from reviews and complaints so the company can improve service and reduce churn.
- AI Customer Support: A 24×7 GenAI chatbot answers policy questions, summarises long claim descriptions for agents, and drafts professional response emails, reducing support workload.

Approach:

Step 1 – Data Collection & Understanding

- Download the datasets listed in the Data Set section (all are free, public datasets from Kaggle).
- Understand each dataset: rows, columns, data types, target variable.

Step 2 – Module 1: Machine Learning (Tabular Data)

- Part A – Regression (Premium Prediction): Perform EDA on the Medical Insurance Cost dataset (age, sex, BMI, children, smoker, region, charges). Handle encoding of categorical columns and feature scaling.
 - Train at least 3 models: Linear Regression, Random Forest Regressor, XGBoost/Gradient Boosting. Compare using R^2 , MAE, RMSE and select the best model.
- Part B – Classification (Fraud Detection): Use the insurance claims fraud dataset. Handle class imbalance (SMOTE or class weights).
 - Train Logistic Regression, Random Forest, and XGBoost classifiers. Compare using Accuracy, Precision, Recall, F1-score and Confusion Matrix.

- Save the best models using pickle/joblib for use in the Streamlit app.

Step 3 – Module 2: Deep Learning (ANN)

- Build an ANN (Keras/TensorFlow) with 2–3 hidden layers for the premium prediction task (regression) and/or fraud detection task (classification).
- Use appropriate activation functions (ReLU for hidden layers; linear for regression output / sigmoid for classification output), optimizer (Adam), loss function, EarlyStopping, and Dropout.
- Plot training vs validation loss curves and compare the ANN results with the ML models of Module 1 in a small comparison table.

Step 4 – Module 3: CNN (Image Classification)

- Use the Car Damage Detection image dataset (two classes: damaged / whole).
- Preprocess images: resize (e.g., 224×224), normalise, and apply data augmentation (rotation, flip, zoom).
- Approach 1: Build a simple CNN from scratch (Conv2D → MaxPooling → Flatten → Dense).
- Approach 2 (recommended): Use Transfer Learning with a pre-trained model like MobileNetV2 or VGG16 and compare with the scratch CNN.
- Evaluate using accuracy and confusion matrix; save the best model (.h5 / .keras).

Step 5 – Module 4: RNN / LSTM (Text Classification)

- Use an insurance/customer review dataset (or any product review dataset relabelled as customer feedback) with text and sentiment labels.
- Preprocess text: lowercase, remove punctuation/stopwords, tokenise, pad sequences.
- Build an Embedding → LSTM → Dense model for binary sentiment classification (positive/negative).
- Evaluate with accuracy and F1-score; test the model with your own sample sentences (e.g., “My claim was settled very fast” → Positive).

Step 6 – Module 5: Generative AI, Prompt Engineering & LLM API

- Get a free/low-cost API key (Groq, Google Gemini, or OpenAI) and store it safely in a .env file (never hard-code it).
- Build three GenAI features using API calls:
 - Policy Q&A Chatbot – answers common insurance questions using a well-crafted system prompt that restricts the bot to insurance topics only.

- Claim Summariser – takes a long claim description and returns a short structured summary (incident, date, damage, estimated severity).
- Email Drafter – generates a professional claim status email to the customer given claim details.
- Demonstrate at least 4 prompt engineering techniques and document them: (1) Zero-shot, (2) Few-shot with examples, (3) Role/Persona prompting (“You are a senior insurance claims officer...”), (4) Chain-of-thought / step-by-step instruction, and structured output prompting (ask for JSON).
- Compare the output quality of a weak prompt vs an engineered prompt and include screenshots in the documentation.

Step 7 – Integration & Deployment

- Build a Streamlit app with a sidebar menu: Home, Premium Prediction, Fraud Detection, Damage Detection (image upload), Review Sentiment, and InsureAI Chatbot.
- Load saved models and connect each page to the corresponding model/API.
- (Optional bonus) Deploy on Streamlit Community Cloud or Hugging Face Spaces.

Skill Up. Level Up

Results:

By the end of this project, learners should achieve:

- A working ML regression model for premium prediction (target $R^2 \geq 0.75$) and a classification model for fraud detection (target F1-score ≥ 0.70 on the minority class after imbalance handling).
- An ANN model with a documented comparison against the classical ML models.
- A CNN/transfer-learning model that classifies vehicle damage images with $\geq 85\%$ test accuracy.
- An LSTM sentiment model with $\geq 85\%$ test accuracy on customer reviews.
- A GenAI chatbot, claim summariser and email drafter powered by LLM API calls, with a short prompt-engineering report showing before/after prompt improvements.
- One integrated Streamlit application demonstrating all five modules, plus complete documentation.

Project Evaluation metrics:

- Regression: R^2 Score, MAE, RMSE.
- Classification (Fraud & Sentiment): Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC-AUC.
- Deep Learning models: Training/validation loss & accuracy curves, overfitting handling (Dropout/EarlyStopping).
- CNN: Test accuracy, confusion matrix, sample predictions on unseen images.
- GenAI Module: Correctness and relevance of responses, quality of prompts (documented techniques), safe API key handling, error handling for API failures.
- Application: Working Streamlit app, code quality (modular functions, comments), GitHub repository with README, and clarity of final presentation/demo.

Technical Tags:

Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, TensorFlow, Keras, ANN, CNN, Transfer Learning, RNN, LSTM, NLP, Sentiment Analysis, Generative AI, LLM, Prompt Engineering, OpenAI/Gemini/Groq API, Streamlit, GitHub, Insurance Analytics, Fraud Detection.

Data Set:

- **Module 1 & 2 (Regression / ANN):** Medical Insurance Cost Dataset – Kaggle (“Medical Cost Personal Datasets”, insurance.csv, ~1,300 rows, CSV). Columns: age, sex, bmi, children, smoker, region, charges (target).
- **Module 1 (Classification):** Insurance Claims Fraud Detection Dataset – Kaggle (“Auto Insurance Claims / insurance_claims.csv”, ~1,000 rows, CSV). Target column: fraud_reported (Y/N).
- **Module 3 (CNN):** Car Damage Detection Dataset – Kaggle (damaged vs whole car images, JPEG, ~2,000 images in two class folders).
- **Module 4 (RNN/LSTM):** Insurance company customer reviews dataset – Kaggle (e.g., insurance reviews / customer feedback dataset with review text + rating). If unavailable, any labelled review dataset may be used and treated as customer feedback (convert ratings: ≥ 4 = Positive, ≤ 2 = Negative).

- **Module 5 (GenAI):** No training dataset required. Uses a live LLM API (Groq / Google Gemini free tier or OpenAI). Learners create a small FAQ text of 10–15 insurance Q&A pairs to use inside few-shot prompts.

Data Set Explanation:

- **Medical Insurance Cost data:** Each row is a policyholder. “charges” is the yearly medical/premium cost to predict. Preprocessing: encode sex/smoker/region (Label/One-Hot Encoding), scale numeric features, check outliers in BMI and charges.
- **Fraud claims data:** Each row is a filed claim with customer, policy, incident and vehicle details; fraud_reported is the target. Preprocessing: drop leakage/ID columns, encode categoricals, handle missing values (“?” entries), and balance classes with SMOTE or class_weight.
- **Car damage images:** Folder-structured image data (train/validation, classes: damaged / whole). Preprocessing: resize to 224×224, rescale pixel values to 0–1, augmentation using ImageDataGenerator / tf.data.
- **Review text data:** Free-text customer reviews with ratings. Preprocessing: clean text, tokenise with Keras Tokenizer (vocab ~10,000), pad sequences to fixed length (e.g., 100), create binary sentiment labels.
- **GenAI FAQ content:** A small self-created knowledge text (policy types, claim process, documents needed). It is injected into the prompt (context) so the chatbot gives grounded, insurance-specific answers.

Project Deliverables:

- Complete source code: Jupyter notebooks for each module (EDA + training) and Python scripts for the Streamlit app.
- Saved model files (.pkl for ML, .h5/.keras for DL/CNN/LSTM).
- Prompt Engineering Report (1–2 pages): the prompts used, techniques applied, and before/after output comparison screenshots.
- GitHub repository with a clear README (project overview, setup steps, screenshots, results table).

- Final project documentation/PPT: problem, approach, results of all 5 modules, and learnings.
- A short demo video or live demo of the Streamlit application.

Project Guidelines:

- Follow PEP-8 coding standards; use meaningful variable names and comments.
- Use virtual environments and a requirements.txt file.
- Never commit API keys — use .env with python-dotenv and add .env to .gitignore.
- Use Git/GitHub from day one; commit after each module with clear messages.
- Keep each module in a separate notebook/script; keep the Streamlit app modular (one function per feature).
- Always split data into train/test (and validation for DL); never evaluate on training data.
- Handle errors gracefully in the app (invalid inputs, API failures, missing image).
- Cite dataset sources and any referenced code/tutorials.

Timeline:

- Week 1: Data collection, EDA, Module 1 (ML – regression + classification), Module 2 (ANN) and Module 3 (CNN with transfer learning).
- Week 2: Module 4 (LSTM sentiment) and Module 5 (GenAI + prompt engineering + LLM API) , Streamlit integration, testing, documentation, GitHub, demo and final evaluation.

PROJECT DOUBT CLARIFICATION SESSION (PROJECT AND CLASS DOUBTS)

About Session: The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

Timing: Monday–Friday: **3:00 PM – 4:00 PM**; Saturday: **2:00 PM – 3:00 PM**

Session link : [☰ Project Doubt Session \[DS/AIML\]](#)

For BMAI project/class topic doubt slot clarification session:

Session link : [☰ BMAI/BADM_Project Doubt Clarification Session Link](#)

Timing: Tuesday, Thursday and Saturday (3:00PM to 4:00PM)

LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

About Session: The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.

Note: This form will Open only on Saturday (after 2 PM) and Sunday on Every Week

Timing:

Monday–Saturday: **11:00 AM – 12:00 PM**; Monday–Friday: **5:00 PM – 6:00 PM**

Booking link : <https://forms.gle/1m2Gsro41fLtZurRA>

Evaluation Metrics : [Project Live Evaluation](#)